# Chapter 9: Graph Algorithms

Dr. Sirasit Lochanachit

# Outline

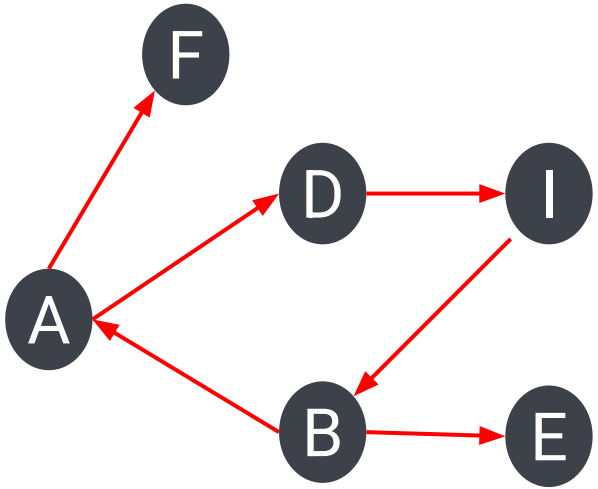
Graphs

- Definition, elements and types
- Graph Representation

Graph Algorithms

- Traversal
  - Depth-first
  - Breadth-first
- Shortest Path

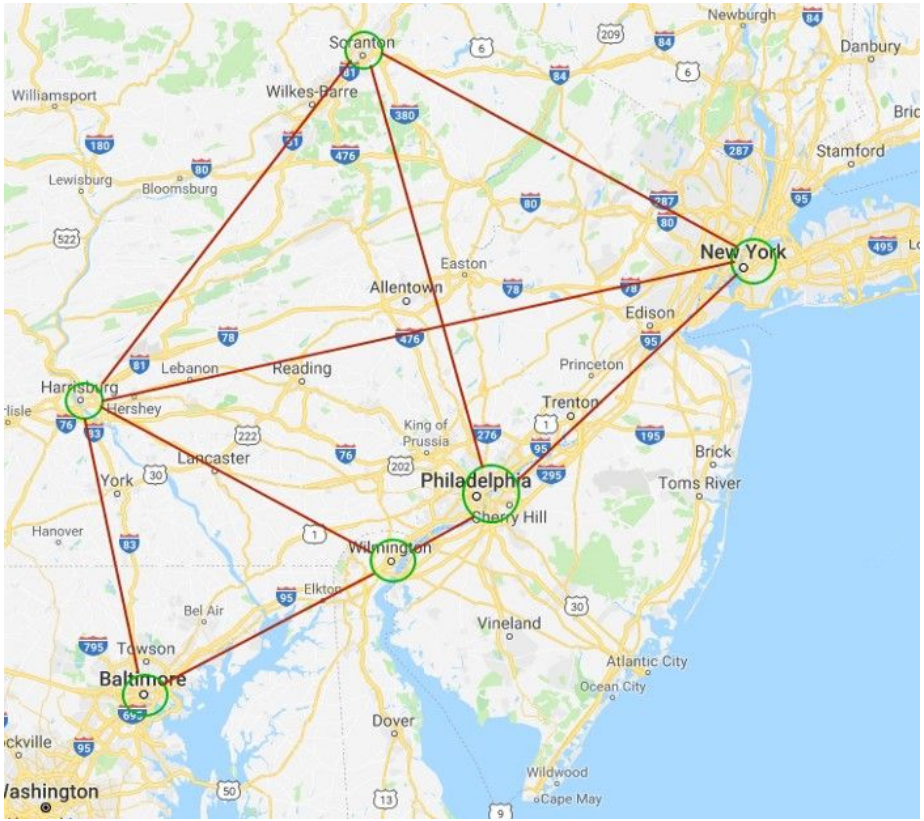
# What is a Graph?




- A **graph** is a set of objects, called vertices or **nodes**, where the actual data is stored and a collection of connections between them, called **edges** or arcs[1].
- A graph can be used to represent relationships between pairs of objects.

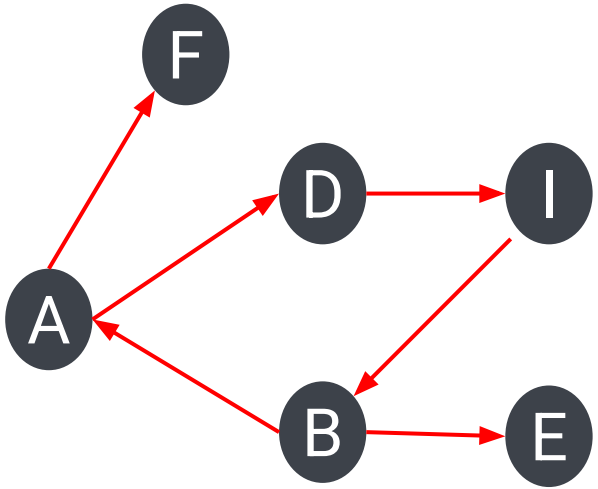
[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

# Graphs




Ref: https://miro.medium.com/max/700/1*2jL0bKbT8ttskAlEVDv1yg.jpeg

# The Basic Elements of Graph



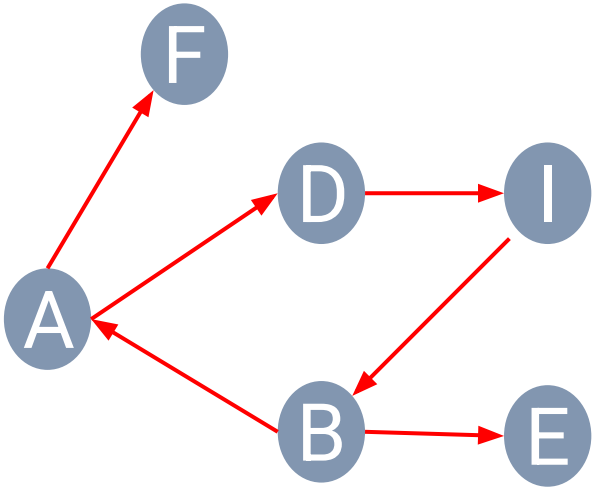

- Formally, a graph *G* is a set *V* of vertices and a collection *E* of pairs of vertices, called edges[1].

V = {A, B, D, E, F, I}

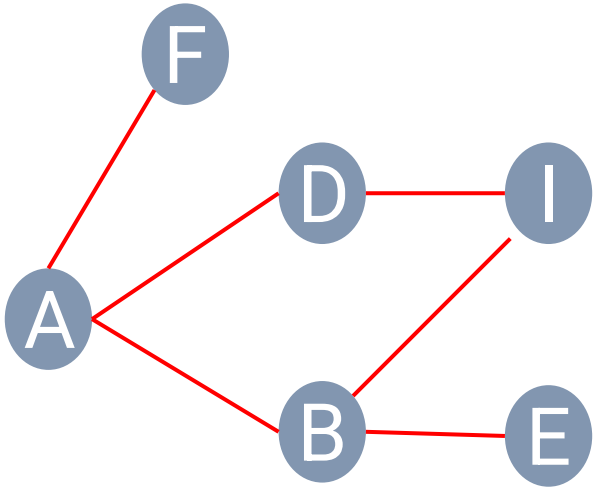
E = {(A, F), (A, D), (B, A), (D, I), (I, B), (B, E)}

[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

# Graph Types



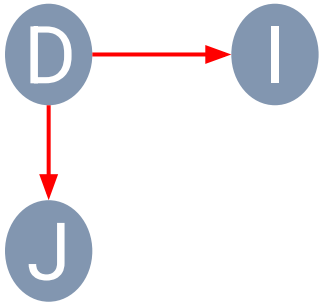

Directed




Undirected

- An edge $(u, v)$ is directed from $u$ to $v$ if the pair $(u, v)$ is ordered.
- An edge $(u, v)$ is undirected if the pair $(u, v)$ is not ordered.

# Graph Terminology

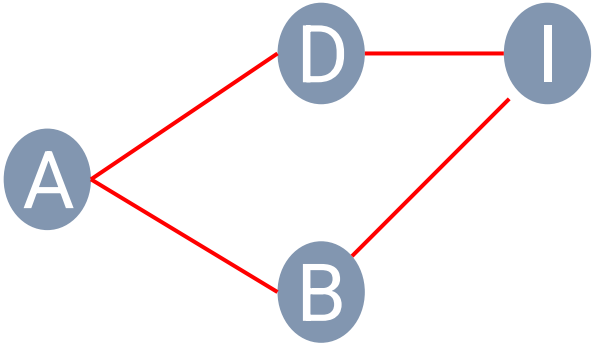



V = {D, I, J}

E = {(D, I), (D, J)}

- **Endpoints**: Two nodes (*u*, *v*) that are joined by an edge.
  - These two nodes are **adjacent**.
- **Origin**: First endpoint (*u*) on a directed edge.
- **Destination**: Second endpoint (*v*) on a directed edge.

# Graph Terminology



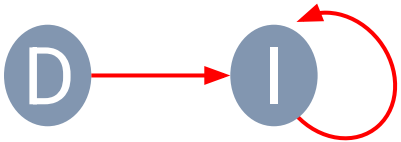

Example Path: A, D, I, B, A

- A **path** is a sequence of nodes and edges that starts at a node and ends at a node such that each node is adjacent to the next one[2].
- Formally, a path is a sequence of nodes $V_1$, $V_2$, $V_3$, …, $V_n$ where $(V_1, V_2)$, $(V_2, V_3)$, …, $(V_{n-1}, V_n)$ ϵ E.

[2] Bradley N. Miller, David L. Ranum, Problem Solving with Algorithms and Data Structures using Python, 2011

# Graph Terminology

- A **loop** is a special case of path where two endpoints are the same.
    - An edge that starts and ends with the same node.



# Graph Algorithms

- Traversals
  - Depth-first traversal
  - Breadth-first traversal
- Minimum Spanning Tree
  - Prim-Jarnik Algorithm
  - Kruskal's Algorithm
- Shortest Path
  - Dijkstra Algorithm
- Etc.

# Graph Traversals

- A **traversal** is a systematic procedure for exploring a graph by examining all of its nodes and edges.
- Graph traversal algorithms are key to answering many fundamental questions about graphs involving the notion of **reachability**, that is, in determining how to travel from one node to another while following paths of a graph[1].
- Two efficient graph traversal algorithms: **depth-first traversal** and **breadth-first traversal**.

[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

# Graph Traversals

**Which algorithm should we use?**

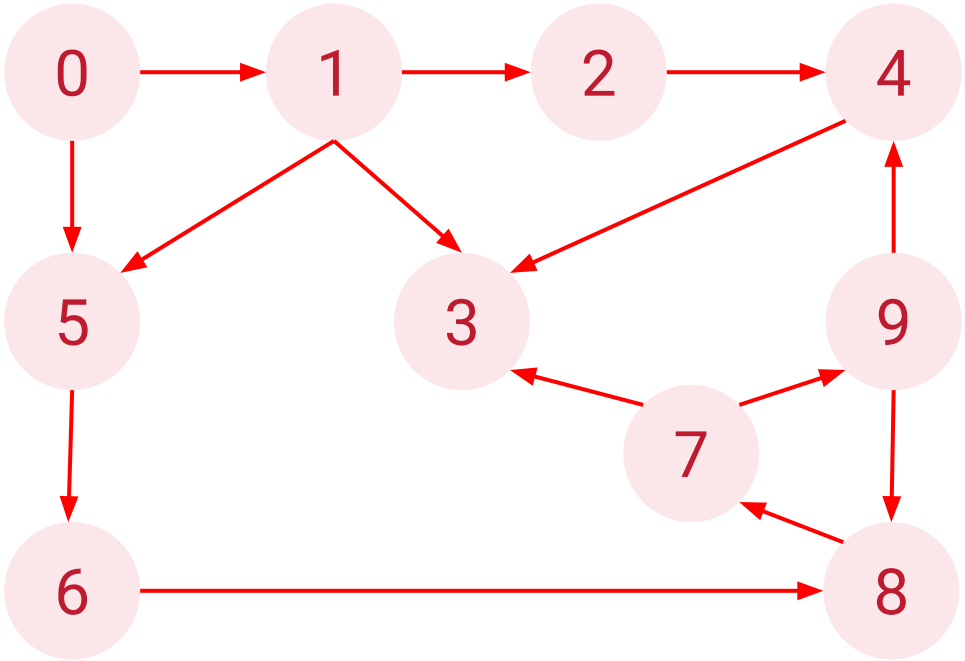
**1) Depth-first traversal (One direction first)**

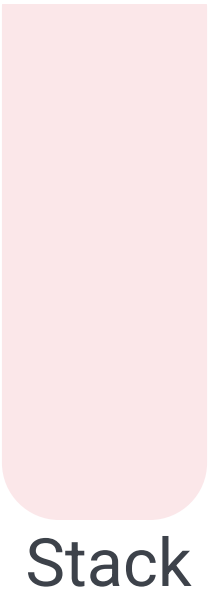
**2) Breadth-first traversal (All direction first)**

# Depth-First Traversal (DFS)

- DFS explores a graph by going as deep as possible along each branch before backtracking.
- Use recursion or a **stack** to keep track of nodes to visit.

# Depth-First Traversal with Stack



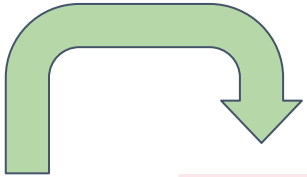




Visited: () - using set or list

Result: [ ]

# Depth-First Traversal with Stack

0



(None, 0)

0. Push the start node along with 'None' as its parent

Stack is used to store parent-child relationships

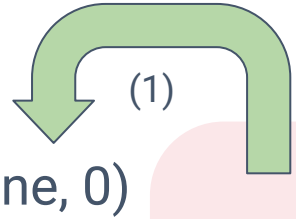
Stack
(parent, child)

Visited: ()

Result: [ ]

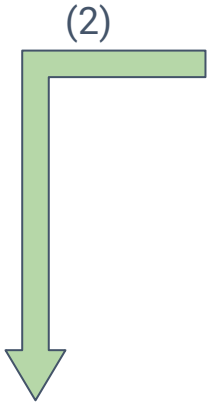
# Depth-First Traversal with Stack

1. Pop the most recently added node and its parent
   - Then check if the node has been visited?
2. If not, then add node to the visited set/list
3. Then record the traversal edge (parent-child), except the first node (0, None).




(None, 0)

0

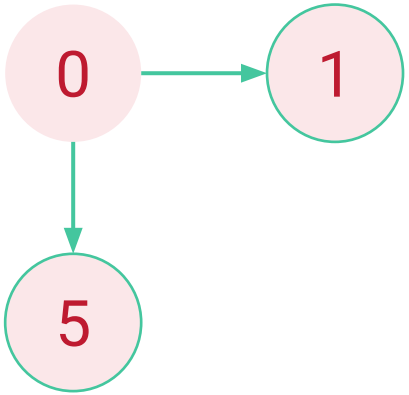



Visited: (0)


Result: [ ]

Stack
(parent, child)

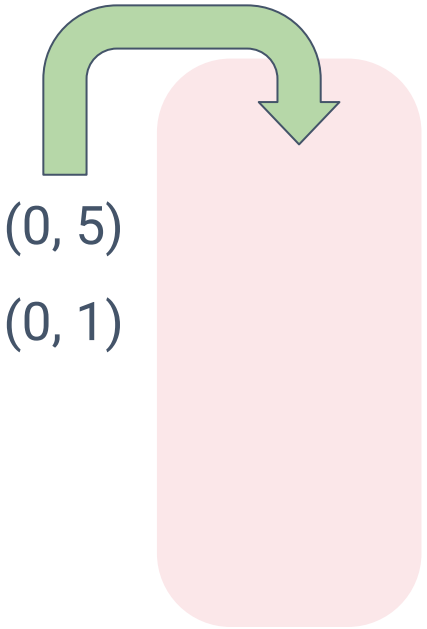
# Depth-First Traversal with Stack




4. Get the neighbors of the current node and sort them by name/number to ensure correct traversal order.

Example: neighbors = [1, 5]

5. Push unvisited neighbors onto the stack with the current node as their parent

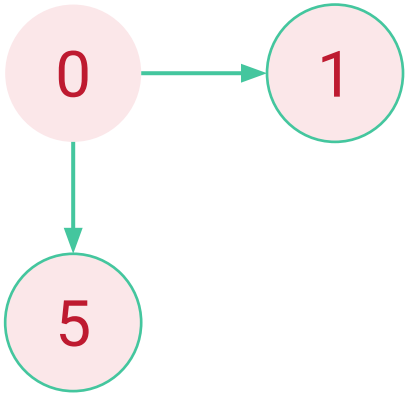



Stack
(parent, child)

Visited: (0)

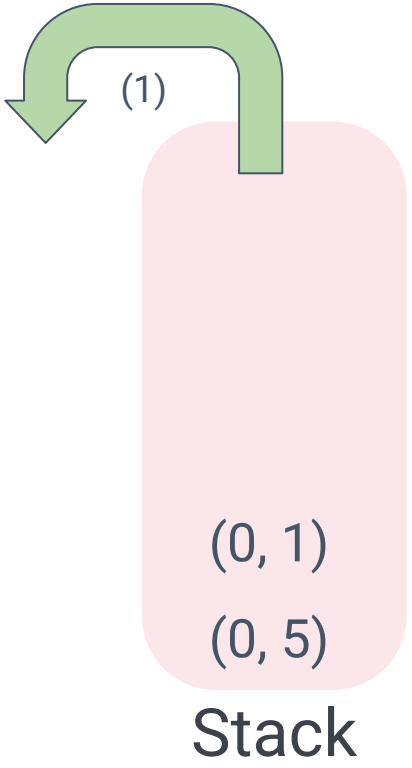
Result: [ ]

# Depth-First Traversal with Stack




Pop the most recently added node and its parent

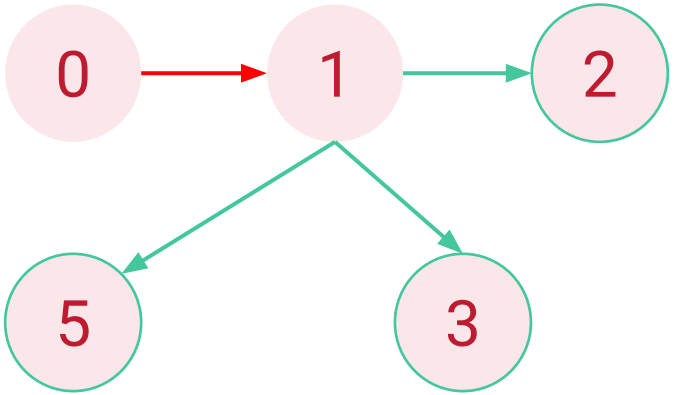
i.e. Redo step 1 to 5.




Visited: (0, 1)

Result: [0 -> 1]

# Depth-First Traversal with Stack




Parent-child (0, 1) is not pushed in a Stack

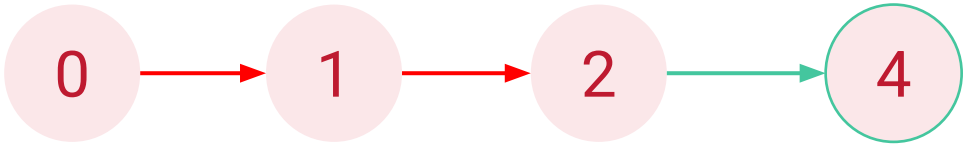
(1, 2)
(1, 3)
(1, 5)
(0, 5)

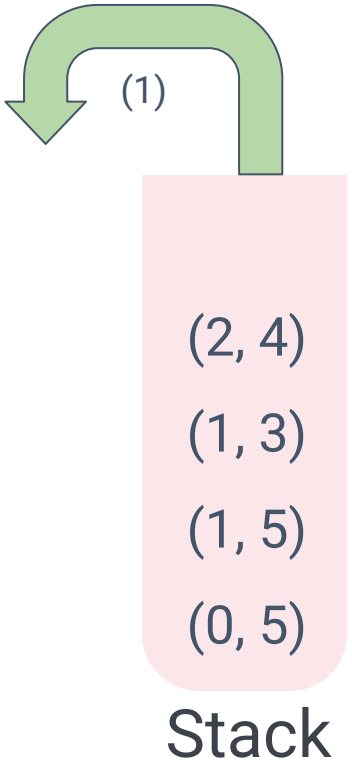
Stack

Visited: (0, 1)

Result: [0 -> 1]

# Depth-First Traversal with Stack

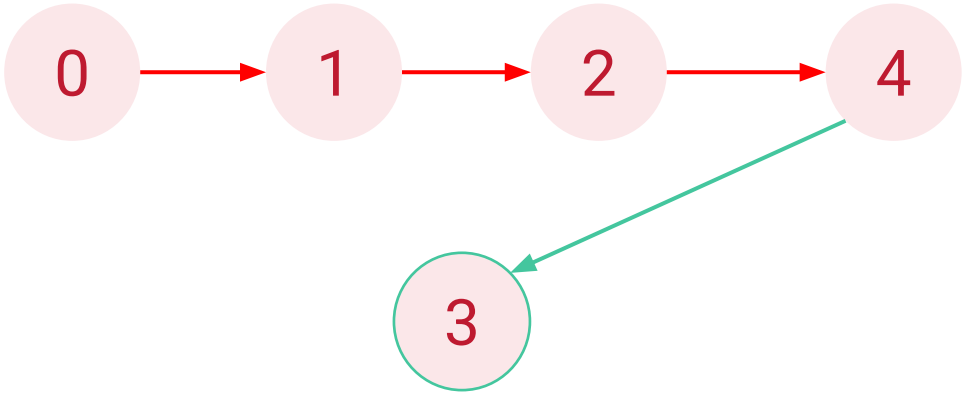


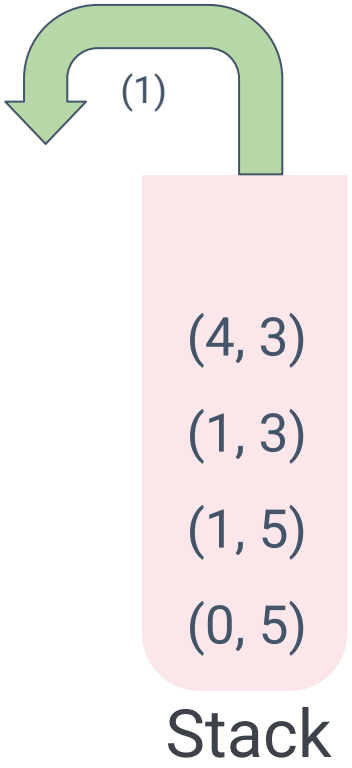




Visited: (0, 1, 2)

Result: [0 -> 1 -> 2]

# Depth-First Traversal with Stack

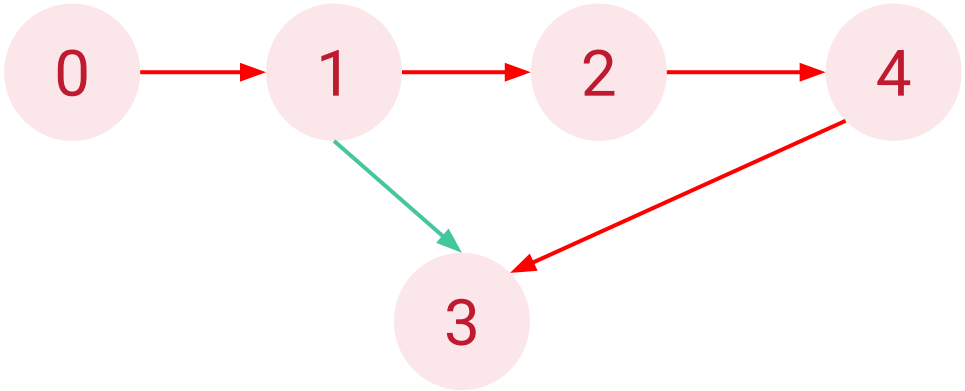


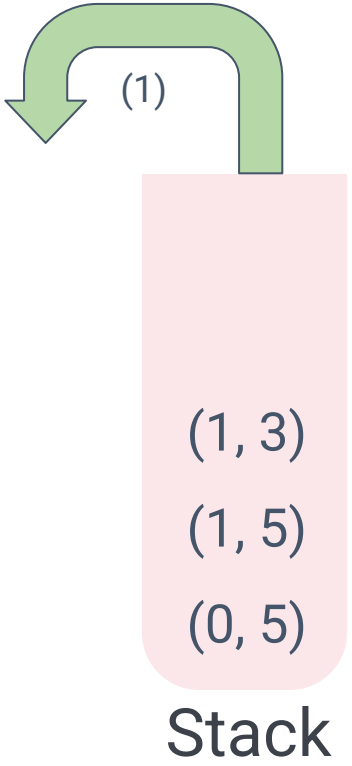




Visited: (0, 1, 2, 4)

Result: [0 -> 1 -> 2 -> 4]

# Depth-First Traversal with Stack



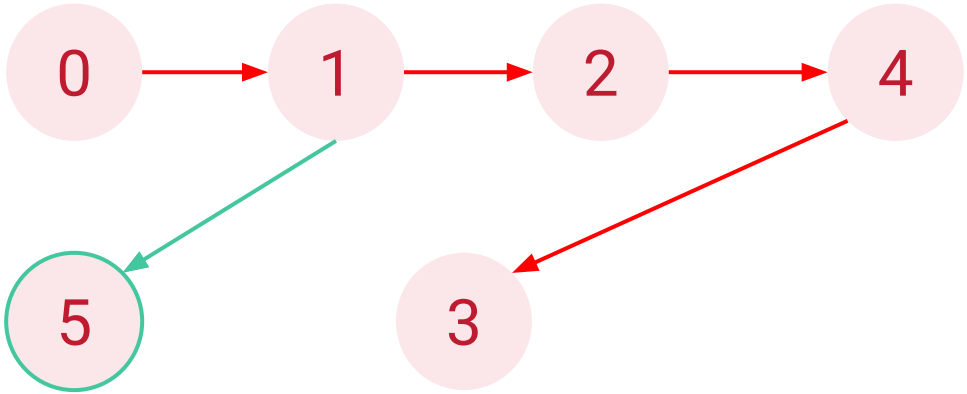



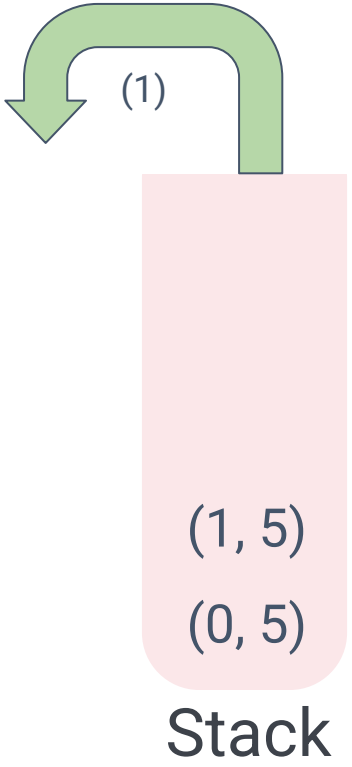

3 is already visited

Visited: (0, 1, 2, 4, 3)

Result: [0 -> 1 -> 2 -> 4 -> 3]

# Depth-First Traversal with Stack

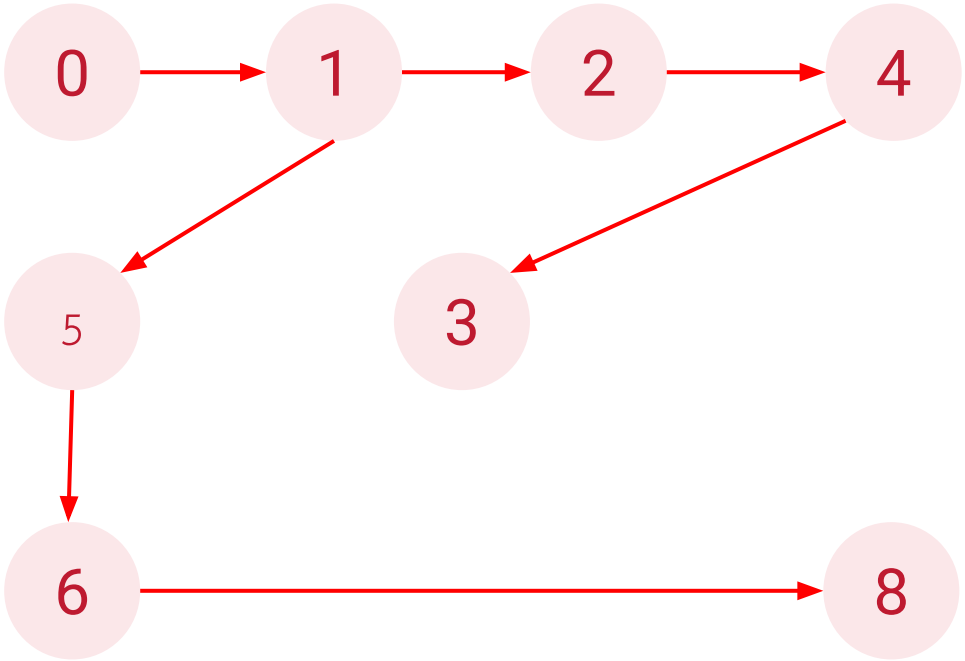


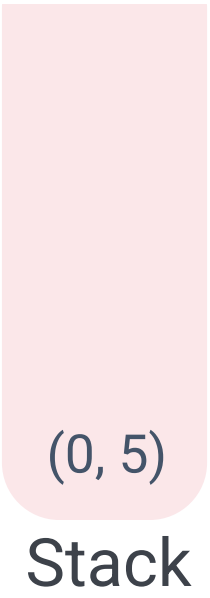




Visited: (0, 1, 2, 4, 3, 5)

Result: [0 -> 1 -> 2 -> 4 -> 3 -> 5]

# Depth-First Traversal with Stack

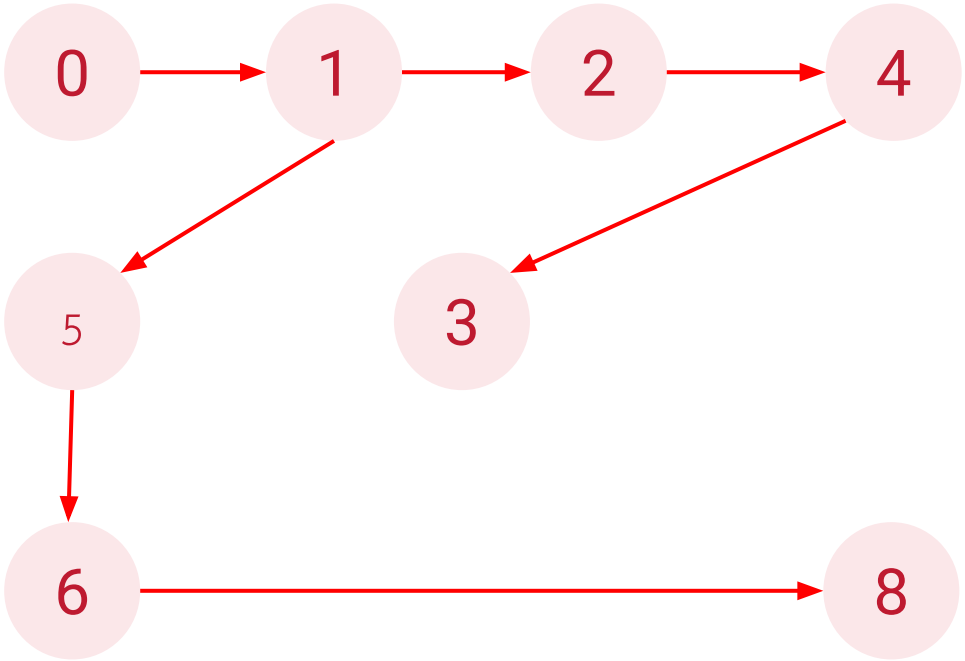


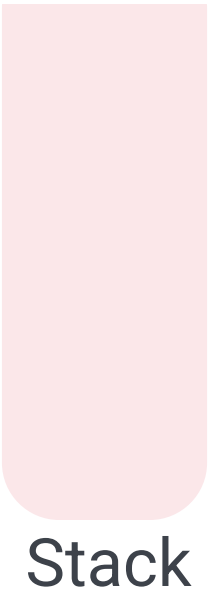




Visited: (0, 1, 2, 4, 3, 5, 6, 8)

Result: [0 -> 1 -> 2 -> 4 -> 3 -> 5 -> 6 -> 8]

# Depth-First Traversal with Stack



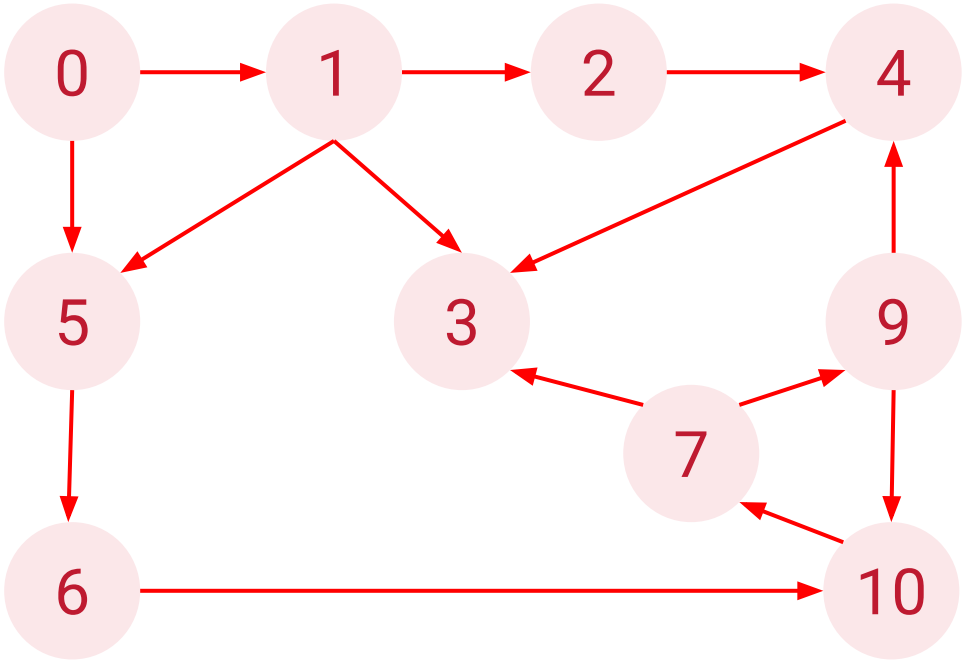

Visited: (0, 1, 2, 4, 3, 5, 6, 8

Result: [0 -> 1 -> 2 -> 4 -> 3 -> 5 -> 6 -> 8

# Breadth-First Traversal (BFS)

- BFS explores a graph level by level, visiting all neighbors of a node before moving to the next level.
- Use a **queue** to keep track of nodes to visit.

# Breadth-First Traversal with Queue




Queue

(parent, child)

Visited: ()

Result: [ ]

# Breadth-First Traversal with Queue

0

Queue

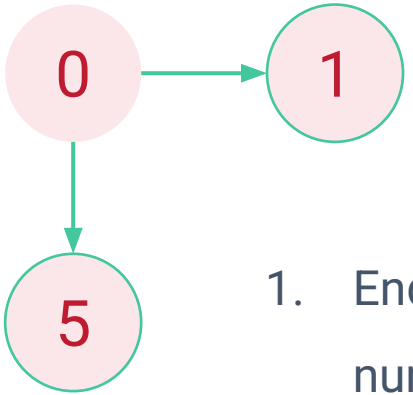
(parent, child)

0. Add the start node to the visited set/list.

Visited: (0)

Result: [ ]

# Breadth-First Traversal with Queue




(0, 1), (0, 5)

1. Enqueue the neighbor nodes in numerical/alphabetical order that are not in visited set/list.

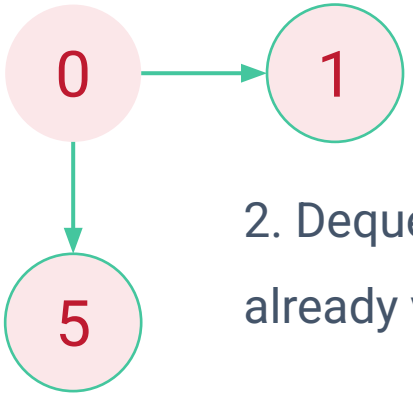
Queue

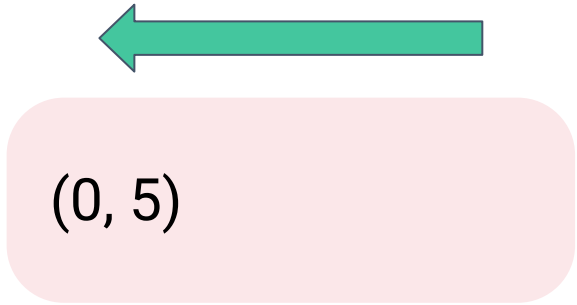
(parent, child)

Visited: (0)

Result: [ ]

# Breadth-First Traversal with Queue




2. Dequeue the next node and check if it is already visited:

If not, add to the visited set/list.

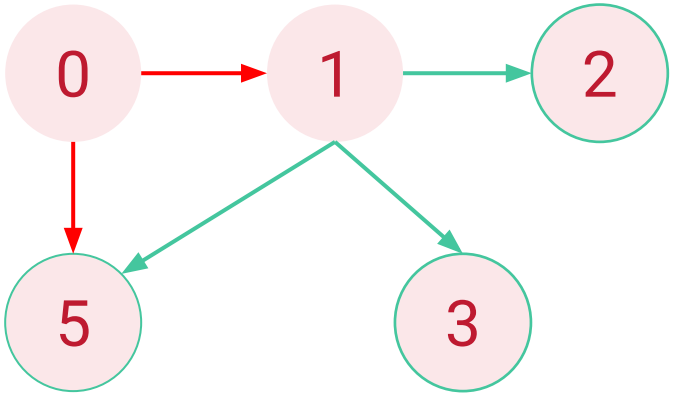
Add parent-child to the result

Then redo step 1.

Visited: (0, 1)

Result: [0 -> 1]

# Breadth-First Traversal with Queue




(0, 5), (1, 2), (1, 3), (1, 5)

Queue

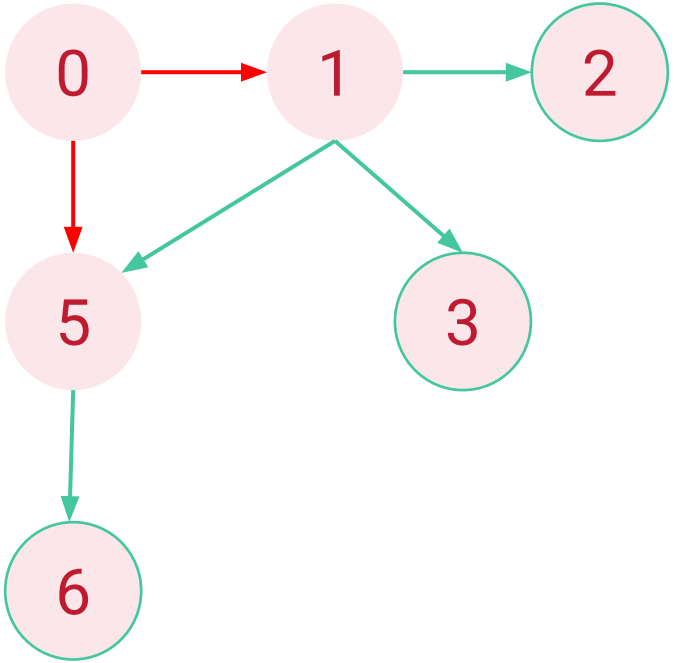
(parent, child)

1. Enqueue the neighbor nodes in numerical/alphabetical order that are not in visited set/list.

Visited: (0, 1)

Result: [0 -> 1]

# Breadth-First Traversal with Queue




(0, 5)

(1, 2), (1, 3), (1, 5), (5, 6)

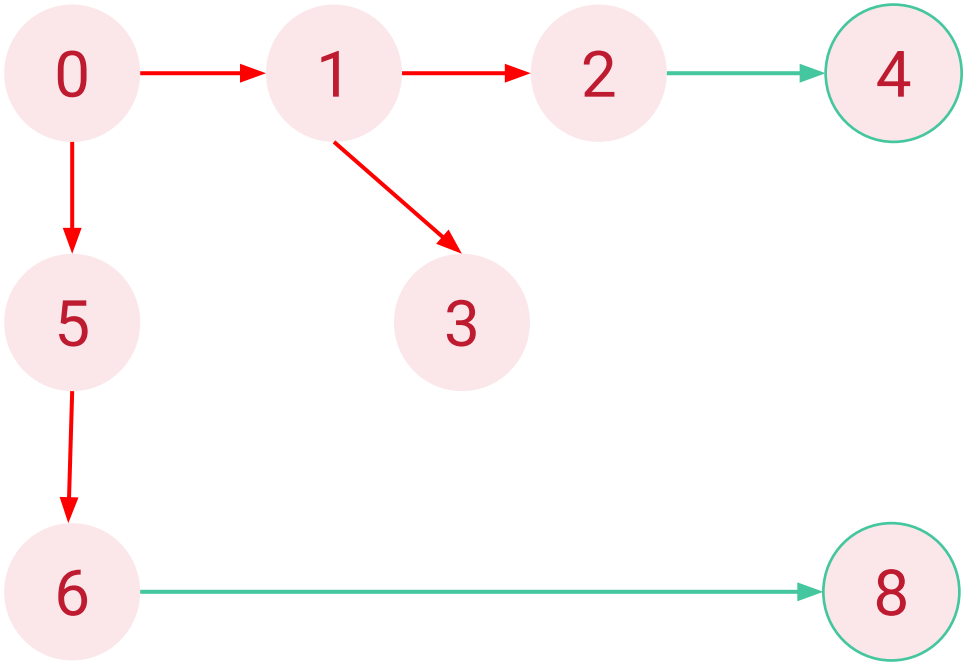
Queue

(parent, child)

Visited: (0, 1, 5)

Result: [0 -> 1 -> 5]

# Breadth-First Traversal with Queue



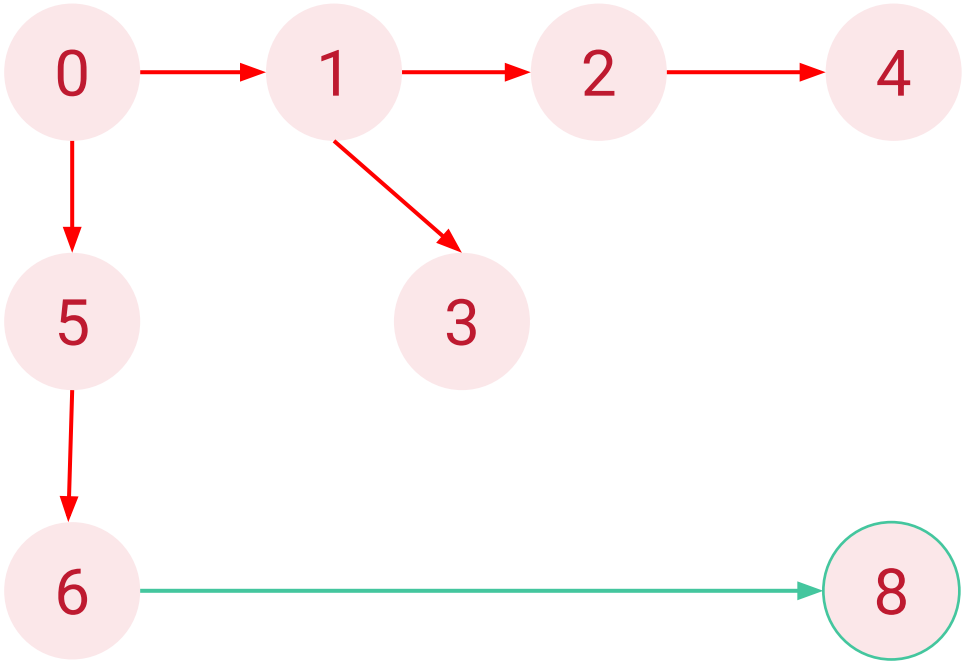

(2, 4), (6, 8)

Queue

Visited: (0, 1, 5, 2, 3, 6)

Result: [0 -> 1 -> 5 -> 2 -> 3 -> 6]

# Breadth-First Traversal with Queue




(2, 4)

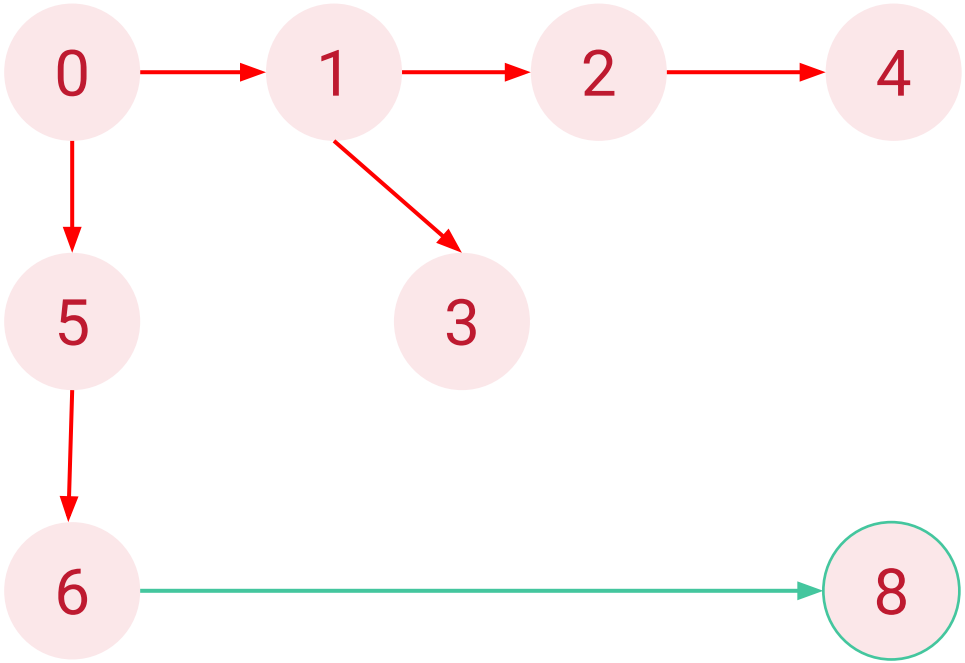
(6, 8)

Queue

Visited: (0, 1, 5, 2, 3, 6, 4)

Result: [0 -> 1 -> 5 -> 2 -> 3 -> 6 -> 4]

# Breadth-First Traversal with Queue



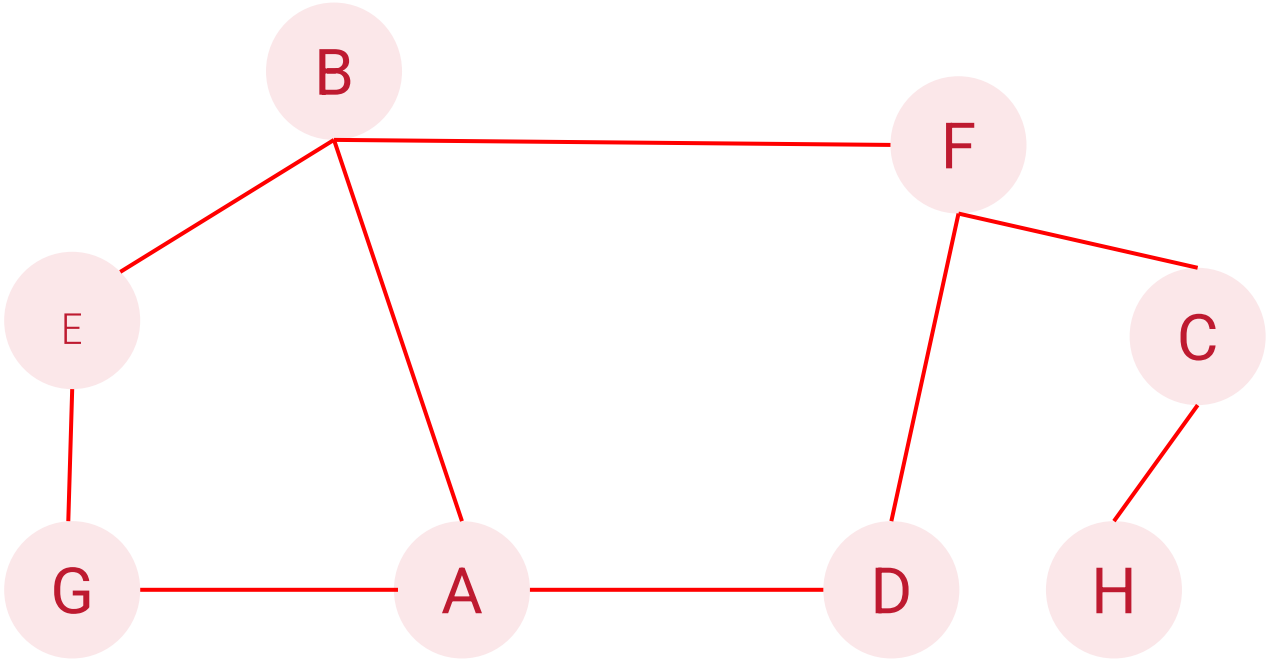

Queue

Visited: (0, 1, 5, 2, 3, 6, 4

Result: [0 -> 1 -> 5 -> 2 -> 3 -> 6 -> 4

# Depth-First and Breadth-First Exercise

Rule: Access node in ascending order (A-Z)

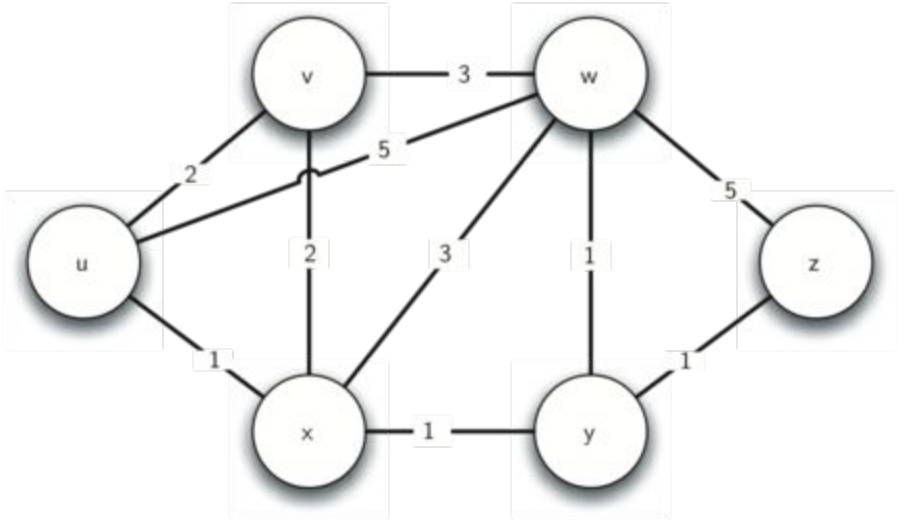


# Graph Traversals

- On undirected and directed graph with $n$ nodes and $m$ edges.
  - A DFS traversal can be performed in $O(n + m)$ time.
  - A BFS traversal can be conducted in $O(n + m)$ time

# Shortest Path Problem

- The network of routers can be represented as a graph with weighted edges.

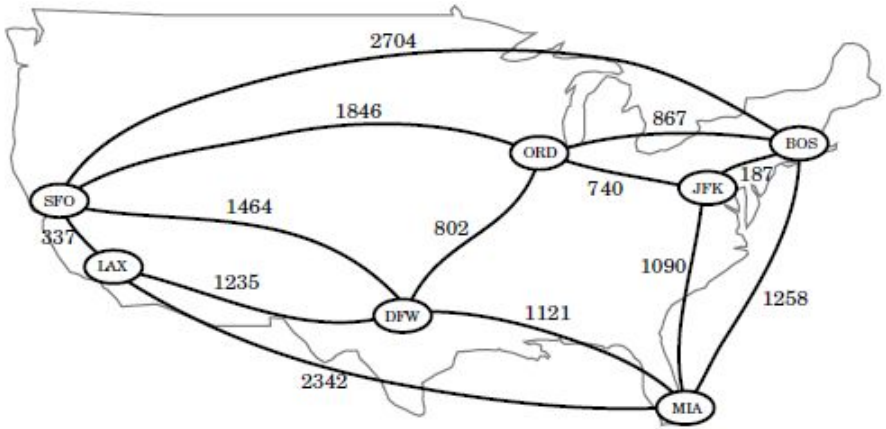


# Shortest Path Problem

- The breadth first strategy can be used to find a **shortest path** from some starting node to every other node in a connected graph.
  - This approach is suitable in cases where each edge is equal to others.
  - However, for other situations, this approach is not efficient.
- It is natural, therefore, to consider graphs whose edges are not weighted equally.

# Weighted Graphs

- A **weight graph** is a graph that has a numeric label *w*(*e*) associated with each edge *e*, called the **weight** of edge *e*.
- For *e* = (*u*, *v*), *w*(*u*, *v*) = *w*(*e*).
- Such weights might represent:
  - Costs
  - Lengths
  - Capacities
  - etc.



# Defining Shortest Paths in a Weighted Graph

- Let *G* be a weighted graph.
- The **length** (or **weight**) of a **path** is the sum of the weights of the edges of *P*.
  - P = $((v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k))$
  - Length of P, denoted w(P) is defined as $$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}).$$
- The distance from a node *u* to a node *v* in *G*, denoted *d*(*u*, *v*) is the length of a minimum-length path (also called **shortest path**) from *u* to *v*.

# Shortest Paths Algorithms

- Shortest path in a graph with all equal weights can be solved with **breadth-first** traversal algorithm.
- Distance cannot be arbitrarily low negative numbers.
  - For instance, the weight of edges represent the cost to travel between cities. If someone pay you to go between the cities, the cost would be negative.
  - Edge weights in *G* should be nonnegative (that is, *w*(*e*) >= 0) for each edge.

# Dijkstra's Algorithm

- An iterative algorithm that provides the shortest path from one starting node to all other nodes in the graph[2].
- Apply **greedy method** to solve the problem by repeatedly selecting the best choice from among those available in each iteration.
  - Useful for optimising cost function over a collection of objects.
- "Weight" breadth-first search starting at the source node *s*.
- Used in link-state routing protocols in computer network.

[2] Bradley N. Miller, David L. Ranum, Problem Solving with Algorithms and Data Structures using Python, 2011

# Dijkstra's Algorithm

```
1  function Dijkstra(Graph, source):
2
3      create vertex set Q
4
5      for each vertex v in Graph:
6          dist[v] ← INFINITY
7          prev[v] ← UNDEFINED
8          add v to Q
10     dist[source] ← 0
11
12     while Q is not empty:
13         u ← vertex in Q with min dist[u]
14
15         remove u from Q
16
17         for each neighbor v of u:           // only v
that are still in Q
18             alt ← dist[u] + length(u, v)
19             if alt < dist[v]:
20                 dist[v] ← alt
21                 prev[v] ← u
22
23     return dist[], prev[]
```

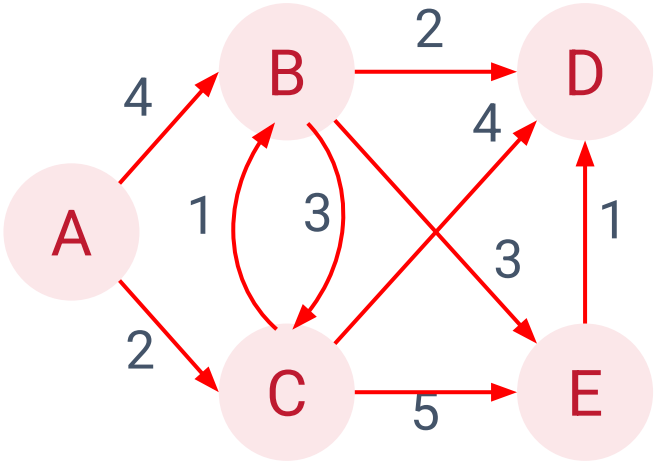
# Dijkstra's Algorithm Variables

- dist[*v*] keeps the shortest/minimum length from the source node *s* for each node *v* in the graph.
  - Initially,
    - dist[*s*] = 0
    - dist[*v*] = Inf for each *v* != s
  - In practice, dist[v] can be set to a very large number than any real distance in the problem.

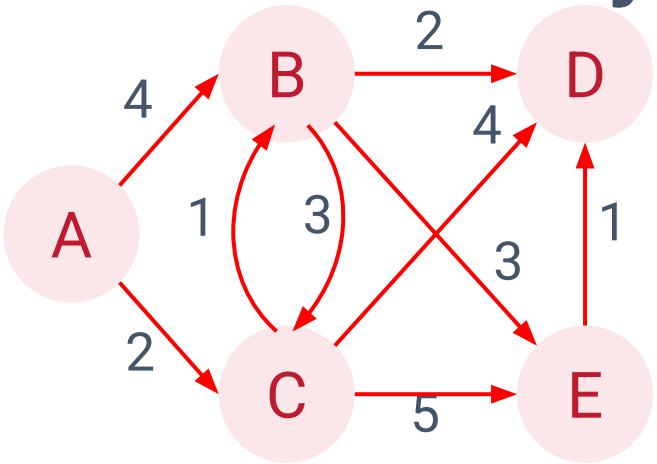
# Dijkstra's Algorithm Variables

- *Q* is a set of all the unvisited nodes, called the unvisited set.
- *prev*[v] is used to keep track of the previous node that provides the shortest path from *s*.

# Dijkstra's Algorithm



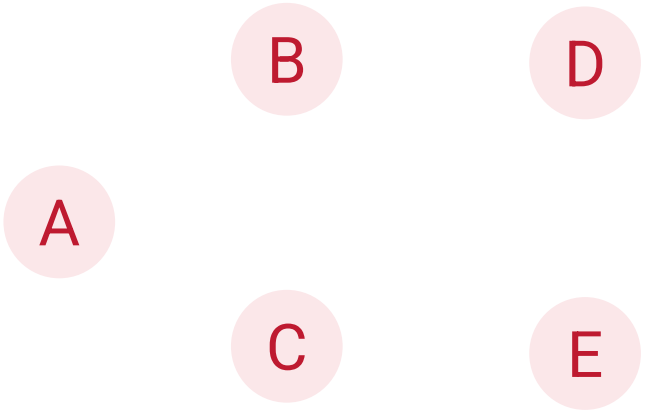
# Dijkstra's Algorithm




Q = {'A', 'B', 'C', 'D', 'E'}

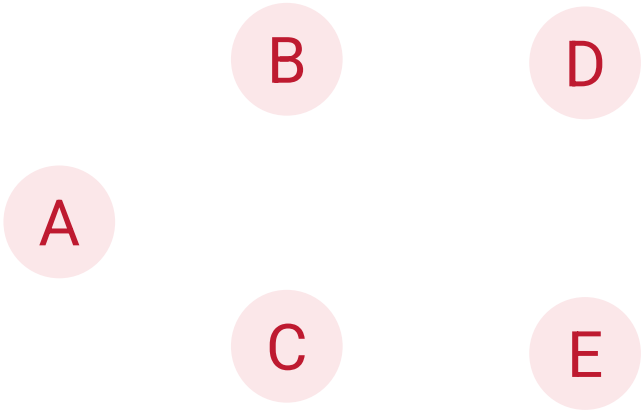
dist = {'A': 0, 'B': ∞, 'C': ∞, 'D': ∞, 'E': ∞}

prev = {'A': None, 'B': None, 'C': None, 'D': None, 'E': None}



# Dijkstra's Algorithm

## Round #0



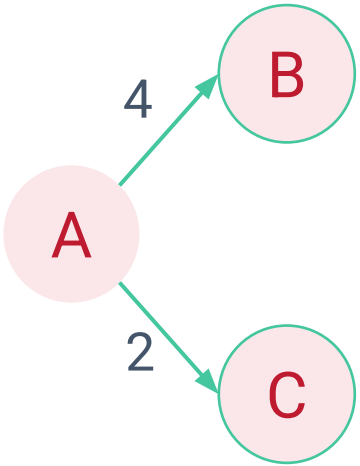

| Node | Path | prev[v] | Cumulative weight / distance (dist[v]) |
|---|---|---|---|
| A | - | None | 0 |
| B | - | None | Inf |
| C | - | None | Inf |
| D | - | None | Inf |
| E | - | None | Inf |

Q = {'A', 'B', 'C', 'D', 'E'}

dist['A'] = 0, dist['B'] = dist['C'] = dist['D'] = dist['E'] = Inf

prev['A'] = prev['B'] = prev['C'] = prev['D'] = prev['E'] = None

# Dijkstra's Algorithm




Q = {"B', 'C', 'D', 'E'}

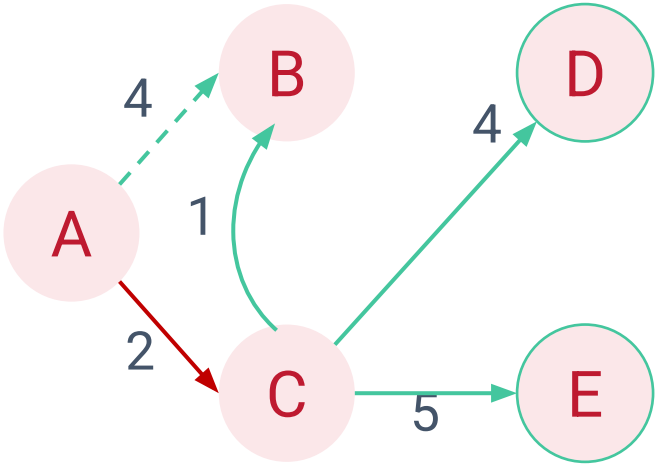
dist['C'] = 2

prev['C'] = 'A'

## Round #1

| Node | Path | prev[v] | Cumulative weight / distance (dist[v]) |
|---|---|---|---|
| A | - | None | 0 |
| B | A -> B | None | 4 |
| C | A -> C | A | 2 |
| D | - | None | Inf |
| E | - | None | Inf |

# Dijkstra's Algorithm

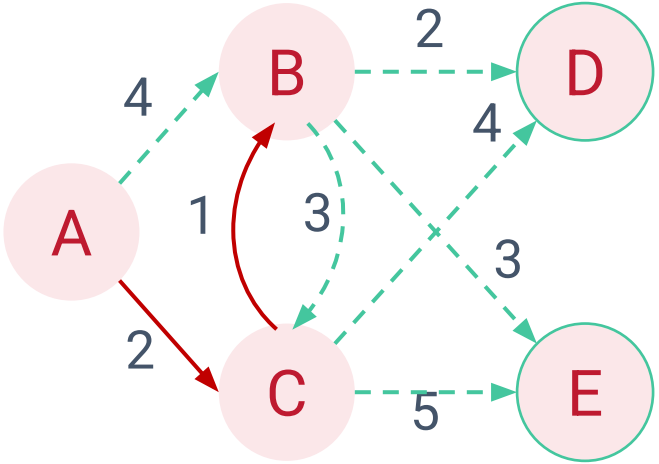



Q = { 'B', 'D', 'E'}
dist['B'] = 3
prev['B'] = 'C'

## Round #2

| Node | Path | prev[v] | Cumulative weight / distance (dist[v]) |
|---|---|---|---|
| A | - | None | 0 |
| B | A -> B | None | 4 |
| | A -> C -> B | C | 2 + 1 = 3 |
| C | A -> C | A | 2 |
| D | A -> C -> D | None | 2 + 4 = 6 |
| E | A -> C -> E | None | 2 + 5 = 7 |

# Dijkstra's Algorithm

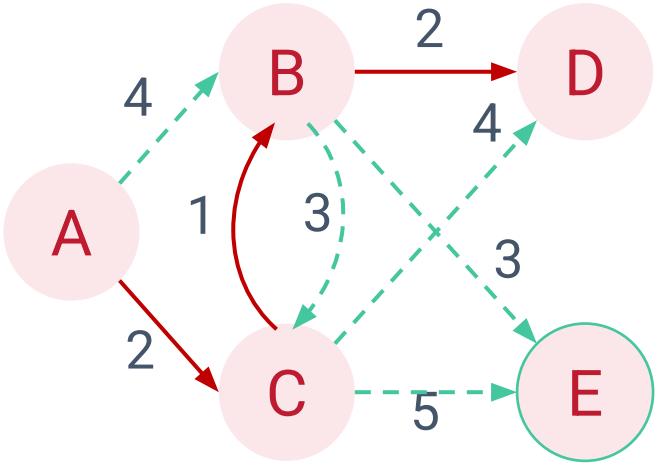



Q = { 'D', 'E'}
dist['D'] = 5
prev['D'] = 'B'

## Round #3

| Node | Path | prev[v] | Cumulative weight / distance (dist[v]) |
|---|---|---|---|
| A | - | None | 0 |
| B | ~~A -> B~~ | ~~None~~ | ~~4~~ |
| | A -> C -> B | C | 3 |
| C | A -> C | A | 2 |
| D | A -> C -> D | None | 2 + 4 = 6 |
| | A -> C -> B -> D | B | 2 + 1 + 2 = 5 |
| E | A -> C -> E | None | 2 + 5 = 7 |
| | A -> C -> B -> E | None | 2 + 1 + 3 = 6 |

# Dijkstra's Algorithm

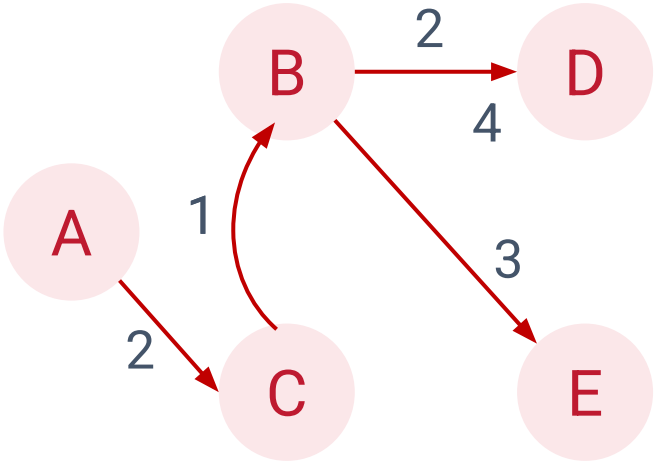



Q = { 'E'}
dist['E'] = 6
prev['E'] = 'B'

## Round #4

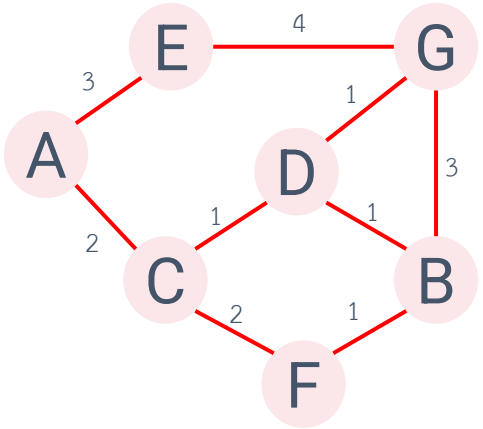
| Node | Path | prev[v] | Cumulative weight / distance (dist[v]) |
|---|---|---|---|
| A | - | None | 0 |
| B | ~~A -> B~~ | ~~None~~ | ~~4~~ |
| | A -> C -> B | C | 3 |
| C | A -> C | A | 2 |
| D | ~~A -> C -> D~~ | ~~None~~ | ~~2 + 4 = 6~~ |
| | A -> C -> B -> D | B | 2 + 1 + 2 = 5 |
| E | ~~A -> C -> E~~ | ~~None~~ | ~~2 + 5 = 7~~ |
| | A -> C -> B -> E | B | 2 + 1 + 3 = 6 |

# Dijkstra's Algorithm




| Node | Path | prev[v] | Cumulative weight / distance (dist[v]) |
|---|---|---|---|
| A | - | None | 0 |
| B | A -> C -> B | C | 3 |
| C | A -> C | A | 2 |
| D | A -> C -> B -> D | B | 5 |
| E | A -> C -> B -> E | B | 6 |

# Dijkstra's Algorithm Exercise




Find the shortest path for each node from node A

Find the distance from A to F

# Dijkstra's Algorithm

- Dijkstra's algorithm works only when the weights are all positive.
  - If there is a negative weight on one of the edges in the graph, the algorithm would never exit.
- Another problem is a complete representation of the graph must be presented for the algorithm to run.
  - Every router has a complete map of all the routers: Not practical.
- Other algorithms allow each router to discover the graph as they go.
  - For instance, distance vector routing algorithm (Computer Networks).
  - Each node computes best path without full view of graph, exchanging link information as they go.